Struktury danych II
drzewa i drzewa BST

Matura rozszerzona z informatyki – Sesja 8

Kacper Gębusia

Plan sesji

1. Drzewa: pojęcia
2. Drzewo binarnych poszukiwań (BST): wstawianie, wyszukiwanie, minimum i maksimum
3. Przechodzenie drzewa: in-order, pre-order, post-order, po poziomach
4. Funkcje rekurencyjne na drzewie
5. Usuwanie węzła i złożoność
6. Ćwiczenia

Powtórzenie z Sesji 7

1. Jaki będzie stan listy `a` i wartości zwrócone przez `pop`?

```
a = [4, 7, 1]
a.append(9)
a.insert(1, 5)
x = a.pop()
y = a.pop(0)
```

2. Co wypisze program?

```
a = [1, 2, 3]
b = a
c = a[:]
b.append(4)
c.append(5)
print(a, b, c)
```

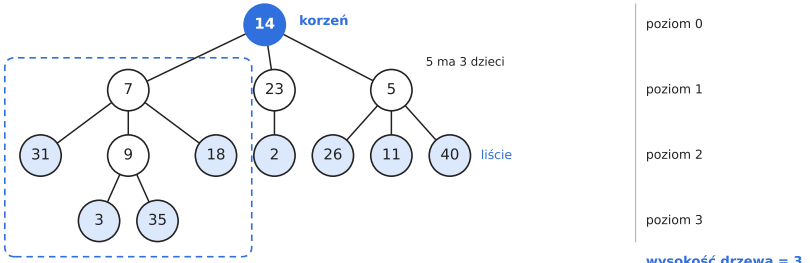
3. Oblicz wartość wyrażenia zapisanego w ONP: `6 2 3 + * 4 -`

4. Kolejka jest pusta. Wykonaj: `enqueue(4)`, `enqueue(9)`, `enqueue(2)`, `dequeue()`, `enqueue(7)`, `dequeue()`, `enqueue(1)`. Co zwróciły kolejne `dequeue`? Jaka jest zawartość kolejki (od początku)?

Drzewo – idea

Drzewo to struktura hierarchiczna zbudowana z **węzłów** połączonych krawędziami, bez cykli. Przykłady: system plików, struktura dokumentu HTML, drzewo genealogiczne, wyrażenie arytmetyczne.

Drzewo ogólne: węzeł może mieć dowolnie wiele dzieci



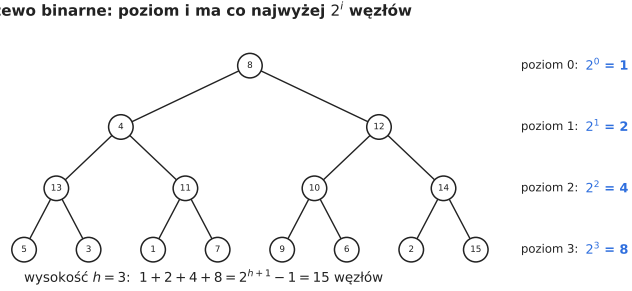
rodzic i dziecko: 7 jest rodzicem węzłów 31, 9, 18; 31, 9, 18 są dziećmi 7

Pojęcia

Pojęcie	Znaczenie
korzeń	jedyny węzeł bez rodzica (wierzchołek drzewa)
liść	węzeł bez dzieci
rodzic / dziecko	węzeł nadrzędny / bezpośrednio podrzędny
poddrzewo	węzeł wraz ze wszystkimi jego potomkami
głębokość (poziom) węzła	liczba krawędzi od korzenia do węzła (korzeń: 0)
wysokość drzewa	największa głębokość węzła (liczba krawędzi najdłuższej ścieżki od korzenia do liścia)

Drzewo binarne

Każdy węzeł ma **co najwyżej dwoje dzieci** – lewe i prawe. Drzewo o wysokości h ma co najwyżej $2^{h+1} - 1$ węzłów (poziom i ma co najwyżej 2^i węzłów).

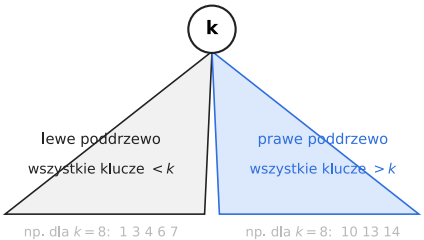


Konwencja w tej sesji: wysokość drzewa z jednym węzłem to 0, a drzewa pustego -1 (dzięki temu wzór `1 + max(...)` działa dla liścia). Na maturze sprawdzamy, czy w poleceniu nie podano innej definicji.

BST – idea

BST (*binary search tree*) to drzewo binarne, w którym dla **każdego** węzła o kluczu k wszystkie klucze w lewym poddrzewie są mniejsze od k, a w prawym większe od k. Zakładamy klucze różne; powtórzony klucz pomijamy.

BST: własność każdego węzła o kluczu k



Reprezentacja węzła – kod

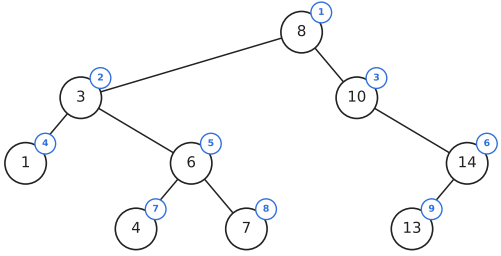
Drzewo reprezentuje wskaźnik na korzeń; puste drzewo to `None`.

```
class Wezel:
    def __init__(self, klucz):
        self.klucz = klucz
        self.lewy = None
        self.prawy = None
```

Wstawianie – idea

Zaczynamy od korzenia: jeśli nowy klucz jest mniejszy – idziemy w lewo, większy – w prawo. Gdy trafimy na puste miejsce (`None`), tam tworzymy nowy węzeł (zawsze jako **liść**).

BST zbudowane z ciągu 8, 3, 10, 1, 6, 14, 4, 7, 13



niebieskie numery: kolejność wstawiania; każdy nowy klucz trafia jako liść

Wstawianie – kod

```
def wstaw(w, k):
    if w is None:
        return Wezel(k)
    if k < w.klucz:
        w.lewy = wstaw(w.lewy, k)
    elif k > w.klucz:
        w.prawy = wstaw(w.prawy, k)
    return w

korzen = None
for k in [8, 3, 10, 1, 6, 14, 4, 7, 13]:
    korzen = wstaw(korzen, k)
```

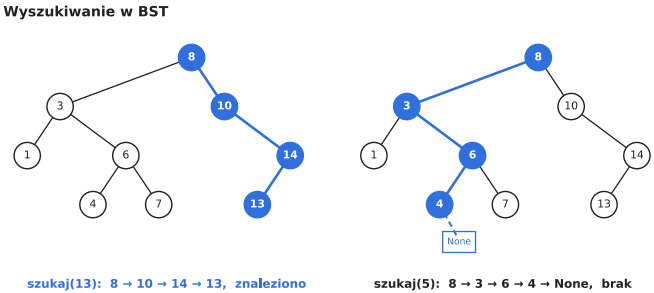
Przebieg budowy dla ciągu $8, 3, 10, 1, 6, 14, 4, 7, 13$

Wstawiany klucz	Ścieżka od korzenia	Miejsce
8	–	korzeń
3	$3 < 8$	lewe dziecko 8
10	$10 > 8$	prawe dziecko 8
1	$1 < 8$, $1 < 3$	lewe dziecko 3
6	$6 < 8$, $6 > 3$	prawe dziecko 3
14	$14 > 8$, $14 > 10$	prawe dziecko 10
4	$4 < 8$, $4 > 3$, $4 < 6$	lewe dziecko 6
7	$7 < 8$, $7 > 3$, $7 > 6$	prawe dziecko 6
13	$13 > 8$, $13 > 10$, $13 < 14$	lewe dziecko 14

Korzeń: 8; liście: 1, 4, 7, 13 (4 liście); wysokość: **3** (np. ścieżka $8 \to 3 \to 6 \to 4$); liczba węzłów: 9; suma kluczy: 66. Tego drzewa używamy w całej sesji.

Wyszukiwanie – idea

Postępujemy jak przy wstawianiu: porównujemy z kluczem w węźle i idziemy w lewo lub w prawo; kończymy po trafieniu lub na `None`.



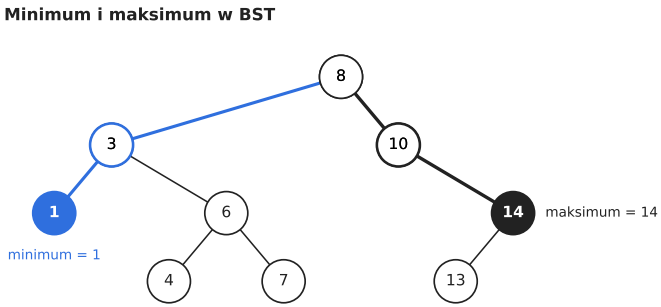
Szukanie 13: 8 → 10 → 14 → 13 (4 węzły, znaleziono). Szukanie 5: 8 → 3 → 6 → 4 i prawe dziecko 4 to `None` – **nie ma** (4 węzły odwiedzone).

Wyszukiwanie – kod

```
def szukaj(w, k):
    if w is None:
        return False
    if k == w.klucz:
        return True
    if k < w.klucz:
        return szukaj(w.lewy, k)
    return szukaj(w.prawy, k)
```

Minimum i maksimum – idea

Najmniejszy klucz leży w **najbardziej lewym** węźle, największy – w **najbardziej prawym**.



W naszym drzewie: `minimum(korzen).klucz` = **1**, `maksimum(korzen).klucz` = **14**.

Minimum i maksimum – kod

```
def minimum(w):
    while w.lewy is not None:
        w = w.lewy
    return w

def maksimum(w):
    while w.prawy is not None:
        w = w.prawy
    return w
```

Przechodzenie drzewa – trzy kolejności

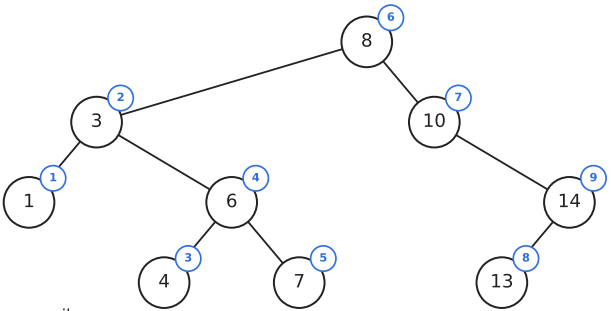
Aby odwiedzić wszystkie węzły, używamy rekurencji. Kolejności różnią się momentem „odwiedzenia” węzła.

Przechodzenie	Kolejność	Zastosowanie
in-order	lewe poddrzewo, **węzeł**, prawe poddrzewo	w BST daje klucze **rosnąco**
pre-order	**węzeł**, lewe, prawe	kopiowanie drzewa, zapis struktury
post-order	lewe, prawe, **węzeł**	usuwanie drzewa, obliczenia „od liści”

In-order – idea

Przechodzenie in-order

lewe poddrzewo → WĘZEŁ → prawe poddrzewo (w BST: klucze rosnąco)



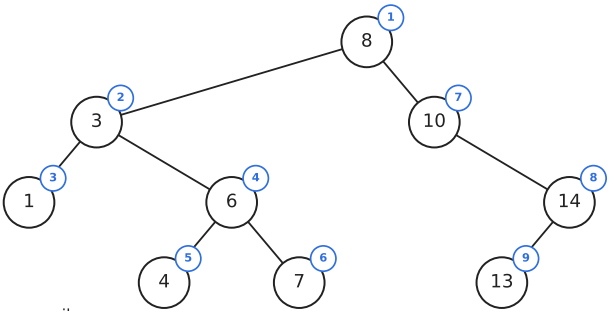
wynik:

1	3	4	6	7	8	10	13	14

Pre-order – idea

Przechodzenie pre-order

WĘZEŁ → lewe poddrzewo → prawe poddrzewo



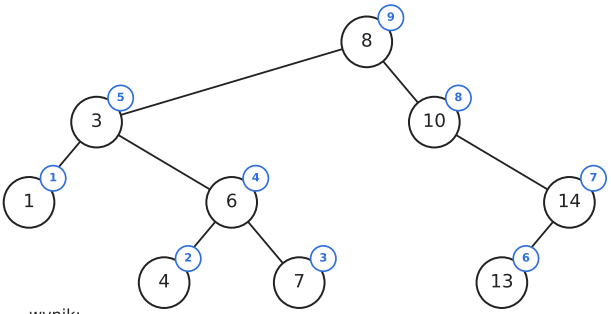
wynik:

8	3	1	6	4	7	10	14	13

Post-order – idea

Przechodzenie post-order

lewe poddrzewo → prawe poddrzewo → WĘZEŁ



wynik:

1	4	7	6	3	13	14	10	8

Przechodzenie rekurencyjne – kod

```
def inorder(w, wynik=None):
    if wynik is None:
        wynik = []
    if w is not None:
        inorder(w.lewy, wynik)
        wynik.append(w.klucz)
        inorder(w.prawy, wynik)
    return wynik
```

```
def preorder(w, wynik=None):
    if wynik is None:
        wynik = []
    if w is not None:
        wynik.append(w.klucz)
        preorder(w.lewy, wynik)
        preorder(w.prawy, wynik)
    return wynik
```

```
def postorder(w, wynik=None):
    if wynik is None:
        wynik = []
    if w is not None:
        postorder(w.lewy, wynik)
        postorder(w.prawy, wynik)
        wynik.append(w.klucz)
    return wynik
```

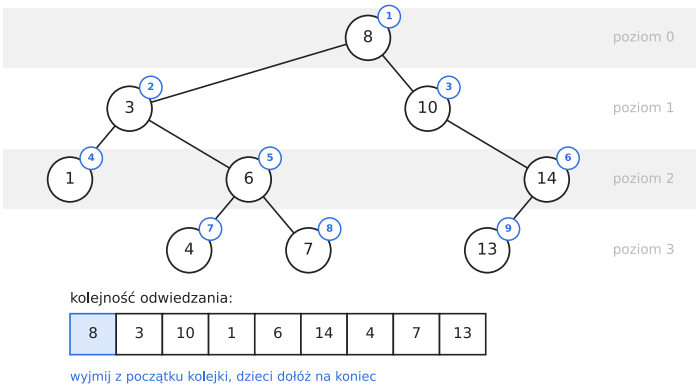
Wyniki przechodzenia dla naszego drzewa

Przechodzenie	Wynik
in-order	1 3 4 6 7 8 10 13 14
pre-order	8 3 1 6 4 7 10 14 13
post-order	1 4 7 6 3 13 14 10 8

Wniosek: wstawienie wszystkich elementów do BST i przejście in-order **sortuje** dane (średnio w czasie $O(n \log n)$, ale duplikaty zostają pominięte).

Przechodzenie po poziomach (BFS) – idea

Odwiedzamy węzły poziom po poziomie, od lewej do prawej. Używamy **kolejki** (Sesja 7): wyjmujemy węzeł, a jego dzieci dokładamy na koniec.



Przechodzenie po poziomach – kod

```
from collections import deque

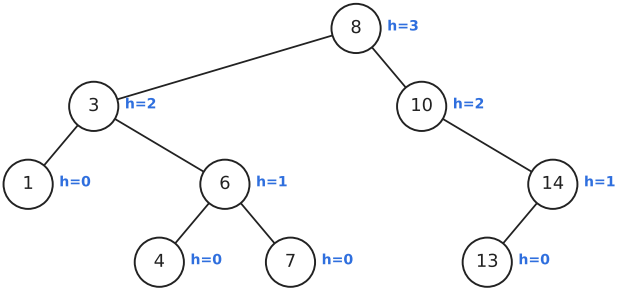
def poziomami(w):
    wynik = []
    if w is None:
        return wynik
    q = deque([w])
    while q:
        x = q.popleft()
        wynik.append(x.klucz)
        if x.lewy:
            q.append(x.lewy)
        if x.prawy:
            q.append(x.prawy)
    return wynik
```

Dla naszego drzewa: **8 3 10 1 6 14 4 7 13** (czyli kolejność wstawiania).

Funkcje rekurencyjne na drzewie – idea

Wynik dla węzła zależy od wyniku dla jego poddrzew.

Rekurencja: wysokość węzła = 1 + max(wysokość lewego, wysokość prawego)



pusty węzeł (None) ma wysokość -1, liść: $1 + \max(-1, -1) = 0$

wysokość drzewa = wysokość korzenia = 3

Funkcje rekurencyjne na drzewie – kod (wysokość, węzły)

```
def wysokosc(w):
    if w is None:
        return -1
    return 1 + max(wysokosc(w.lewy), wysokosc(w.prawy))

def ile_wezlow(w):
    if w is None:
        return 0
    return 1 + ile_wezlow(w.lewy) + ile_wezlow(w.prawy)
```

Funkcje rekurencyjne na drzewie – kod (liście, suma)

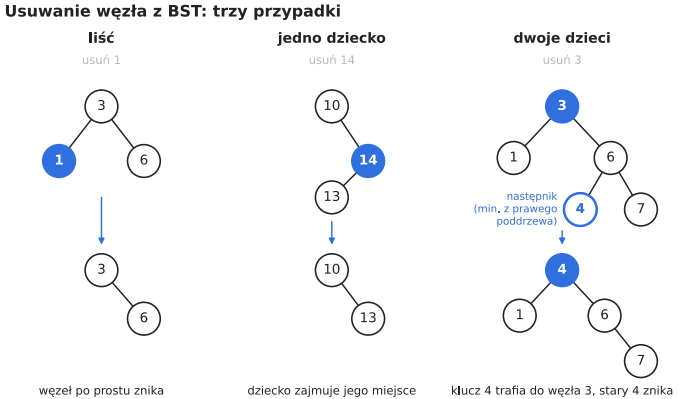
```
def ile_lisci(w):
    if w is None:
        return 0
    if w.lewy is None and w.prawy is None:
        return 1
    return ile_lisci(w.lewy) + ile_lisci(w.prawy)

def suma(w):
    if w is None:
        return 0
    return w.klucz + suma(w.lewy) + suma(w.prawy)
```

Dla naszego drzewa: `wysokosc` = 3, `ile_wezlow` = 9, `ile_lisci` = 4, `suma` = 66. Złożoność każdej: $O(n)$.

Usuwanie z BST – idea

Najpierw znajdujemy węzeł o kluczu *k*. Trzy przypadki: **liść** – usuwamy; **jedno dziecko** – zastępujemy węzeł dzieckiem; **dwoje dzieci** – kopiujemy klucz **następnika** (minimum prawego poddrzewa) i usuwamy następnik.



Usuwanie z BST – kod

```
def usun(w, k):
    if w is None:
        return None
    if k < w.klucz:
        w.lewy = usun(w.lewy, k)
    elif k > w.klucz:
        w.prawy = usun(w.prawy, k)
    else:
        if w.lewy is None:
            return w.prawy
        if w.prawy is None:
            return w.lewy
        m = minimum(w.prawy)                # nastepnik
        w.klucz = m.klucz
        w.prawy = usun(w.prawy, m.klucz)
    return w
```

Usuwanie – przykłady na naszym drzewie

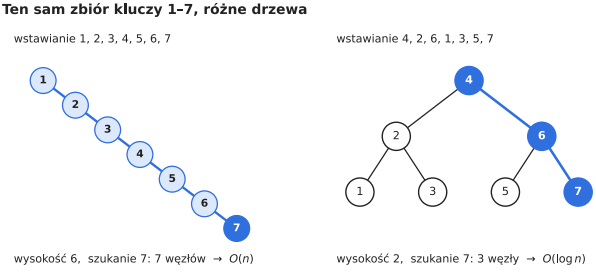
Każdy przykład od drzewa początkowego; podajemy pre-order po usunięciu.

Klucz	Przypadek	Co się dzieje	Pre-order
1	liść	znika	8 3 6 4 7 10 14 13
14	jedno dziecko (13)	13 zajmuje miejsce 14	8 3 1 6 4 7 10 13
3	dwoje dzieci (1 i 6)	następnik: minimum z prawego poddrzewa $= 4$; 4 trafia do węzła 3, stary węzeł 4 znika	8 4 1 6 7 10 14 13
8 (korzeń)	dwoje dzieci	następnik $= 10$ trafia do korzenia, jego prawe dziecko (14) zajmuje miejsce	10 3 1 6 4 7 14 13

W każdym przypadku in-order nadal jest rosnący.

Złożoność operacji

Wstawianie, wyszukiwanie i usuwanie kosztują $O(h)$, gdzie h to wysokość drzewa. Drzewo zrównoważone: $h \approx \log_2 n$, operacje $O(\log n)$. Drzewo zdegenerowane (np. klucze rosnąco): $h = n - 1$, operacje $O(n)$.



Zrównoważenie utrzymują specjalne drzewa (AVL, czerwono-czarne), ale nie są wymagane na maturze.

Przykład – te same 7 kluczy

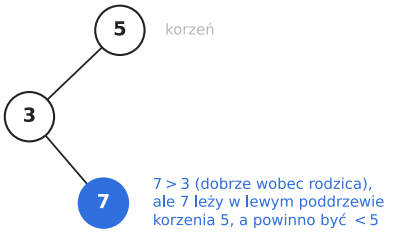
Kolejność wstawiania	Wysokość	Węzłów odwiedzonych przy szukaniu 7
1, 2, 3, 4, 5, 6, 7	**6** (łańcuch)	**7**
4, 2, 6, 1, 3, 5, 7	**2** (drzewo pełne)	**3**

Ile poziomów potrzeba? Drzewo o wysokości h ma maksymalnie $2^{h+1} - 1$ węzłów, więc dla n węzłów minimalna wysokość to najmniejsze h spełniające $2^{h+1} - 1 \geq n$. Dla $n = 1000$: $2^{10} - 1 = 1023 \geq 1000$, więc $h = \mathbf{9}$ – wystarczy 10 porównań, żeby znaleźć klucz wśród 1000.

Sprawdzanie, czy drzewo jest BST – idea

Samo porównanie węzła z jego dziećmi nie wystarcza – klucz musi mieścić się w przedziale wyznaczonym przez **wszystkich** przodków.

Pułapka: samo porównanie z rodzicem nie wystarcza



dopuszczalny przedział dla 7: (3; 5) → czy_bst zwraca False

Drzewo: korzeń 5, lewe dziecko 3, prawe dziecko 3 to 7. Funkcja `czy_bst` zwraca `False`.

Sprawdzanie, czy drzewo jest BST – kod

```
def czy_bst(w, lo=None, hi=None):
    if w is None:
        return True
    if lo is not None and w.klucz <= lo:
        return False
    if hi is not None and w.klucz >= hi:
        return False
    return czy_bst(w.lewy, lo, w.klucz) and czy_bst(w.prawy, w.klucz, hi)
```

Ćwiczenia na zajęciach (1/3)

1. Dla drzewa z prezentacji (wstawianie $8, 3, 10, 1, 6, 14, 4, 7, 13$) podaj: korzeń, liście, rodzica węzła 4, głębokość węzła 7 i wysokość drzewa.
2. Zbuduj BST, wstawiając kolejno: $12, 5, 18, 3, 9, 15, 20, 7, 10$. Podaj liście i wysokość.
3. Podaj przejścia in-order, pre-order i post-order dla drzewa z zadania 2.
4. W którym miejscu drzewa z prezentacji zostaną wstawione klucze 11 oraz 5? Podaj ścieżkę.

Ćwiczenia na zajęciach (2/3)

5. Ile węzłów odwiedzi wyszukiwanie klucza 10 oraz 16 w drzewie z zadania 2?
6. Zbuduj BST z ciągu a) $2, 4, 6, 8, 10$ b) $6, 2, 10, 4, 8$. Podaj wysokości i in-order obu drzew.
7. Pre-order drzewa BST to: $8, 3, 1, 6, 4, 7, 10, 14, 13$. Odtwórz drzewo i podaj jego post-order.

Ćwiczenia na zajęciach (3/3)

8. Usuń z drzewa z konspektu (każdy raz od drzewa początkowego): a) 1 b) 14 c) 3. Podaj pre-order po każdym usunięciu.
9. a) Ile węzłów może mieć co najwyżej drzewo binarne o wysokości 3?
   b) Jaka jest minimalna wysokość drzewa binarnego z 1000 węzłami?
10. Co zwrócą `suma(korzen)` i `ile_lisci(korzen)` dla drzewa z konspektu?

Praca domowa (1/3)

1. Zbuduj BST, wstawiając kolejno: $50, 30, 70, 20, 40, 60, 80, 35$. Podaj przejścia in-order, pre-order i post-order oraz wskaż, gdzie trafił klucz 35.
2. Dla drzewa z zadania 1 podaj wyniki funkcji `minimum`, `maksimum`, `wysokosc` i `ile_wezlow`.
3. Zbuduj BST z ciągu $7, 3, 9, 1, 5, 8, 12, 4, 6$. Podaj in-order, pre-order, post-order, wysokość i liczbę liści.

Praca domowa (2/3)

4. a) Czy ciąg $2, 3, 5, 4, 8$ może być wynikiem przejścia in-order drzewa BST? b) Czy ciąg $5, 3, 2, 4, 8, 7, 9$ może być pre-orderem drzewa BST? Jeśli tak, narysuj drzewo i podaj jego post-order.
5. Podaj kolejność odwiedzania węzłów drzewa z zadania 3 przy przechodzeniu po poziomach (funkcja `poziomami`). Jakiej struktury danych ona używa i dlaczego?
6. Z drzewa z zadania 3 usuń kolejno klucze 3, a potem 7. Po każdym usunięciu podaj pre-order.

Praca domowa (3/3)

7. Porównaj drzewa zbudowane z ciągów $1, 2, 3, 4, 5, 6, 7$ oraz $4, 2, 6, 1, 3, 5, 7$: podaj wysokości i liczbę węzłów odwiedzonych przy szukaniu klucza 7. Co z tego wynika dla złożoności?
8. Napisz funkcję `ile_lisci(w)` (rekurencyjnie). Dla chętnych: napisz `czy_bst(w)` i podaj, co zwróci dla drzewa: korzeń 5, lewe dziecko 3, prawe dziecko 3 równe 7.

Dziękuję za uwagę!